



FACULTY:

FACULTY OF ICT

PROGRAMME:

PROGRAM

YEAR:

2

LECTURER:

Prof. Sanghoon Lee

FROM:

BLESSINGS KATIMBA

COURSE:

OOP

COURSE CODE:

BSCICT 2203

DUE DATE:

16th APRIL 2026

Fixed Code

```
1  #include <stdio.h>
2
3  int main(){
4      int i, j;
5      const int ROW = 3;
6      const int COL = 4;
7
8      int arr[ROW][COL] = {
9          {1, 2, 3, 4},
10         {5, 6, 7, 8},
11         {9, 10, 11, 12}
12     };
13
14     // Using pointer (*) notion
15     printf("Using pointer notation: \n");
16     for(i = 0; i < ROW; i++){
17         for(j = 0; j < COL; j++){
18             printf("%4d", (*(arr + i) + j));
19         }
20         printf("\n");
21     }
22
23     printf("\n");
24
25     //using arrayat [] notation
26     printf("Using array notation: \n");
27     for(i = 0; i < ROW; i++){
28         for(j = 0; j < COL; j++){
29             printf("%4d", arr[i][j]);
30         }
31         printf("\n");
32     }
33
34     return 0;
35 }
```

Code Output:

```
Using pointer notation:
  1   2   3   4
  5   6   7   8
  9  10  11  12

Using array notation:
  1   2   3   4
  5   6   7   8
  9  10  11  12

-----
Process exited after 0.3013 seconds with return value 0
Press any key to continue . . .
```

This program shows how to access and print elements of a two-dimensional array on C using both array indexing ([]) for displaying output (*)

The program begins by including the standard input-output library `stdio.h`, which lets the use of functions like `printf()` for displaying output.

In the 'main' function, two integer variables `i` and `j` are declared. These are used as loop counters to navigate through the rows and columns of the array. Two constants, `ROW` and `COL`, are defined to represent the number of rows (3) and columns (4) in array.

A two-dimensional integer array `arr[ROW][COL]` is initialized with values from 1 till 12 and arranged in a matrix format. This creates the structure of 3 rows and 4 columns.

The program first prints the elements using pointer notation. The expression `*(arr + i) + j` is used to gain access to each element. Nested 'for' loops are used to go through all rows and columns, printing each value in a formatted way.

After that, the program prints the same array with standard array indexing being used. The expression `arr[i][j]` directly accesses the element in the *i*-th and *j*-th column. Both methods produce identical output, demonstrating that array indexing and pointer notation are equivalent in C when working with arrays.